

Lab(期末大作业) SysY 语言编译器设计与实现

Soft Deadline

2026 年 1 月 15 日 23: 59

中期检查

2025 年 12 月 15 日 23: 59

任务描述

• 任务1: SysY 编译器前端设计与实现 (25分)

在《汇编与编译原理》课程的大作业中，你需要组建不超过 4 人的团队，用 C++17 实现 SysY 语言到 LLVM IR 的编译器前端。

你的 C++ 项目能构建名为 `compiler` 的可执行文件，并使用命令行参数指定输入输出文件。例如你要将 `input.sy` 文件编译成 `output.ll`，你需要执行 `./compiler input.sy output.ll`。

除 ANTLR4 以外，你不应该引入其他第三方库（如 LLVM）。如确实有需要引入的第三方库，请联系 谢博帆 助教确认该库是否被允许引入。

• 任务2: 拓展研究 (5分)

你可以以小组形式完成下面任意一个任务来获得分数：

- 了解毕昇编译器，利用毕昇编译器实现代码优化，给出示例并分析代码执行效率。
- 在生成 LLVM IR 的基础上，将其进一步转换成 X86、ARM 或 RISC-V 等指令集下的汇编代码。（需要实现最基本的指令选择和寄存器分配，不能使用现有工具如 Clang）
- 构建控制流图，实现活跃变量分析、常量传播、窥孔优化等机器无关优化。

- 完善你的编译器前端，实现更多语言特性，如浮点数、指针、for循环、结构体、switch语句等。

你无需做得十分完美，例如实现浮点数和指针或实现窥孔优化就能获得大部分分数。如果选择生成汇编代码，只需挑选部分IR指令进行翻译即可。

拓展研究部分以文档形式展现，你需要在[网络学堂](#)上提交你的拓展研究文档（PDF格式）。

下面的内容主要介绍任务1，也就是实现 SysY 语言到 LLVM IR 的编译器前端。

SysY 语言定义

SysY语言是毕昇杯比赛采用的语言，其实质是 C 语言的子集。SysY 语言的词法规则和语法规则与 PA1、PA3 中的定义一致，更多细节请见 PDF: [SysY语言定义](#)。

测试用例中，部分 SysY 语言会调用 sylib 库， sylib 库的实现在仓库

`test/resources/sylib.c` 中。在代码实现时，你可以预先将这些函数加入到符号表中。

LLVM IR

你可以通过下面的材料学习 LLVM IR

- [LLVM 架构设计和原理](#)
- [LLVM IR 基本概念](#)
- [LLVM IR 详解](#)
- [LLVM 前端和优化层](#)

鼓励大家自己查阅资料、通过大语言模型学习 LLVM IR。你也可以阅读 [LLVM 官方文档](#)。

输入输出样例

• 输入样例

注：大作业采用文件读入，输入文件为 SysY 语言源代码，文件名以 `.sy` 结尾。

input.sy

```
int main() {
    int a = 1;
    return a;
}
```

• 输出样例

注：大作业中，你需要将生成的 IR 输出到一个文件中，例如在 `./compiler input.sy output.ll` 命令中，你需要输出到一个名为 `output.ll` 的文件中。

output.ll

```
; ModuleID = 'moudle'
source_filename = "moudle"

define i32 @main() {
mainEntry:
    %a = alloca i32, align 4
    store i32 1, i32* %a, align 4
    %a1 = load i32, i32* %a, align 4
    ret i32 %a1
}
```

注意：我们不要求你输出的 IR 与样例中的完全一致，只要能正确执行并返回正确的结果即可。我们会捕获运行你生成的 IR 时的标准输出以及 `main` 函数的返回值，并与标准答案进行比对。具体评测方法可以看框架代码中的 `run-test.py` 脚本。

框架代码

你需要在 [thss-2025-compiler-final](#) 仓库中获取框架代码。我们推荐你建立小组的代码仓库，并将代码提交到你的小组仓库中。

仓库中包含全部测试用例、本地测试脚本、编译脚本、打包脚本等。但是框架代码中不包含编译器的实现，所以本次大作业可以称得上是“从零编写编译器”的实践。

项目结构：

```
thss-2025-compiler-final
├── CMakeLists.txt      # CMake 配置文件
├── Makefile            # Makefile 文件，包含编译、测试等任务
```

```
├── README.md
├── SysYLexer.g4          # 词法分析规则文件，你需要自己完成（同 PA1）
├── SysYParser.g4        # 语法分析规则文件，你需要自己完成（同 PA3）
├── include               # 存放头文件，Type.h 只是示例
│   └── Type.h
├── lib
│   └── antlr-4.13.1-complete.jar
├── package.py            # 打包脚本，用于将项目提交到 GradeScope
├── run-test.py           # 测试脚本，用于本地测试
├── src
│   └── main.cpp          # 源代码，请合理地组织代码，不要将逻辑写到一个文件中
├── test
│   └── resources
│       ├── functional    # 测试用例
│       ├── libsysy.a
│       ├── sylib.c       # SysY 语言链接库
│       └── sylib.h
└── third_party           # 第三方库
    └── antlr4-runtime
```

运行与测试

大作业采用 CMake 构建项目，在项目根目录中执行下面的命令构建 `compiler`：

```
mkdir build
cd build
cmake ..
make -j8
```

此时，可以看到 `build/` 目录下生成了可执行文件 `compiler`。

接下来，你可以用你自己实现的 `compiler` 来编译 SysY 代码：

```
# 在 build 目录下
./compiler ../test/resources/functional/00_main.sy 00_main.ll
```

生成的 `00_main.ll` 就是你编译出来的 LLVM IR 文件。你可以使用 `clang` 将其进一步编译成二进制文件。

```
clang 00_main.ll -w -o 00_main
./00_main
```

```
# 查看程序返回值
echo $?
```

当你的 SysY 需要链接 sylib 库时，可以使用下面的命令：

```
clang 00_main.ll ../test/resources/sylib.c -w -o 00_main
```

好消息是，在大作业中，你可以在 `test/` 目录下找到所有的测试用例。但为了避免同学们采用识别文件名/识别输入特征的方式来生成 LLVM IR，我们在 OJ 上设置了一个特殊的隐藏用例，这个用例极其简单(只包含 main 函数和一个返回语句)，只有通过这个测试用例才能获得分数。

我们也为你准备了本地的评分脚本 `run-test.py`，本地脚本评测速度远远快于 GradeScope 在线评测，希望大家在提交作业前先进行本地测试。实验保证所有的测试用例均没有语法错误。

```
# 当前在项目根目录中
python run-test.py # 或直接 make test
```

TEST RESULTS	
Test Name	Result
00_main	✓ Passed
01_var_defn2	✓ Passed
02_var_defn3	✓ Passed
03_arr_defn2	✗ Failed
04_arr_defn3	✗ Failed
05_arr_defn4	✗ Failed

提交与评分

无特殊情况，一个小组内所有成员成绩相同。

部分分数通过的测试用例数总测试用例数

大作业分数部分分数中期检查分数分拓展研究分数不超过分

GradeScope显示的分数为 OJ 部分分数的百分制展示。即：

显示分数通过的测试用例数总测试用例数

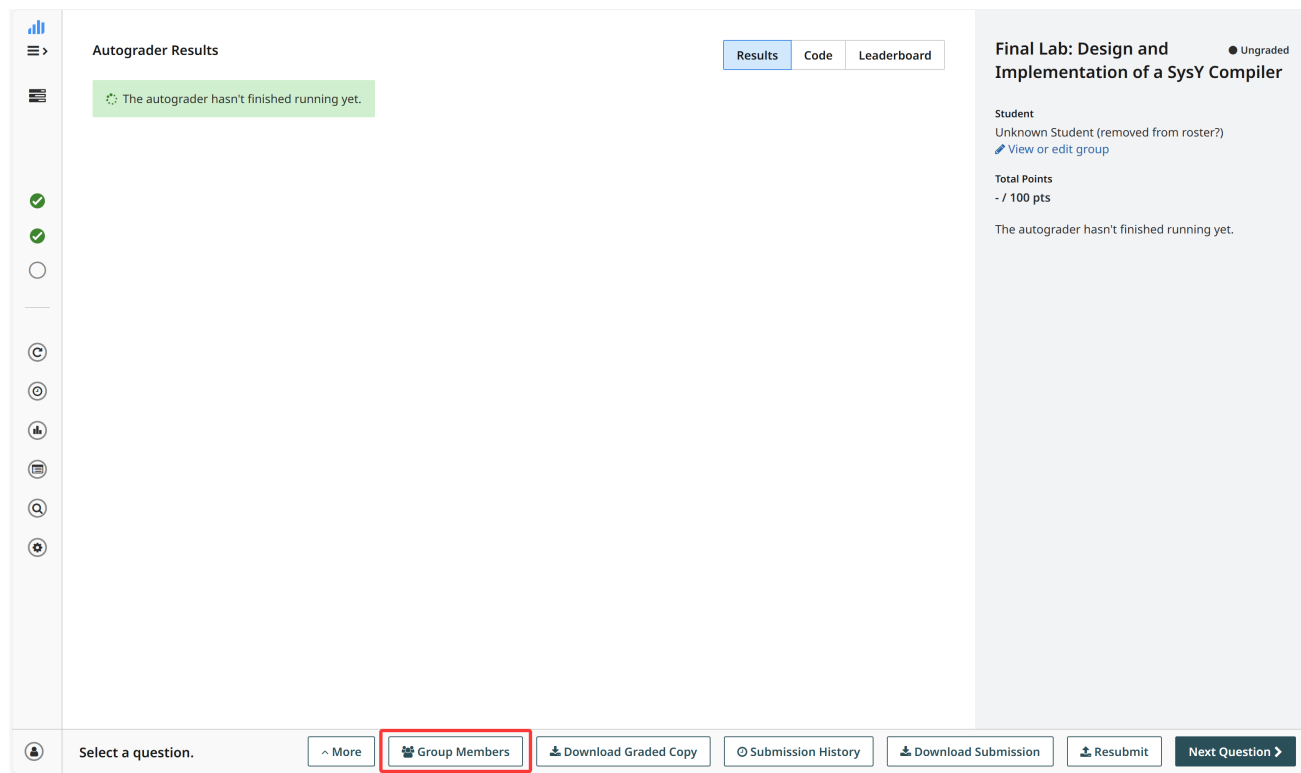
中期检查：中期检查 DDL 时，小组应至少通过 3 个测试用例，即你应当完成 main 函数的翻译和常数表达式计算的翻译。

GradeScope 基础镜像：Ubuntu 22.04。

在 GradeScope 上提交实验代码：

1. **打包文件：**在项目根目录中运行 `python package.py` 打包文件。
2. **提交文件：**在 GradeScope 中提交你的 ZIP 文件，填写排行榜上的组名，排行榜会动态显示当前排名前十名的队伍。在 GradeScope 提交后，将你的组员添加到本次提交中。

！ 注意：每一次提交都要添加组员信息，操作方式见下图：



3. 查看反馈：大作业评测时间较长，提交后耐心等待自动评分结果，Online Judge 会构建 Docker 对你项目进行自动测试，你可以多次提交。注意 GradeScope 可以选择你的历史提交分数，你应当选择你的历史最高分作为成绩。

• 其他要求

1. 你需要使用 git 来管理你小组的代码，请在开发的过程中遵守 git 的提交规范，保留 git 的历史记录。（建议使用 `git commit -s` 来签名你的提交）
2. 为了控制提交大小，请务必通过 `.gitignore` 文件来过滤 build 等产物目录。
3. 请尽量保证软件可扩展性、可维护性。
4. 请务必遵循团队协作规范，确保代码质量，不要做团队杀手！😡。

有余力的小组可以搭建自己的 CI/CD 流水线，但这些都不作为计分项。

实验警告

- 学生不得复制其他小组的代码和拓展研究文档
- 学生不得向其他小组提供代码和拓展研究文档
- 严禁使用识别文件名、识别 SysY 语言输入特征等方法生成 LLVM IR 或针对测试用例做优化
- 严禁在代码中调用其他工具、调用 AI 模型来生成 LLVM IR

大作业指导

• 环境配置

在 PA3 环境配置的基础上，你还需要安装 Clang, 用来将 LLVM IR 编译成二进制文件。

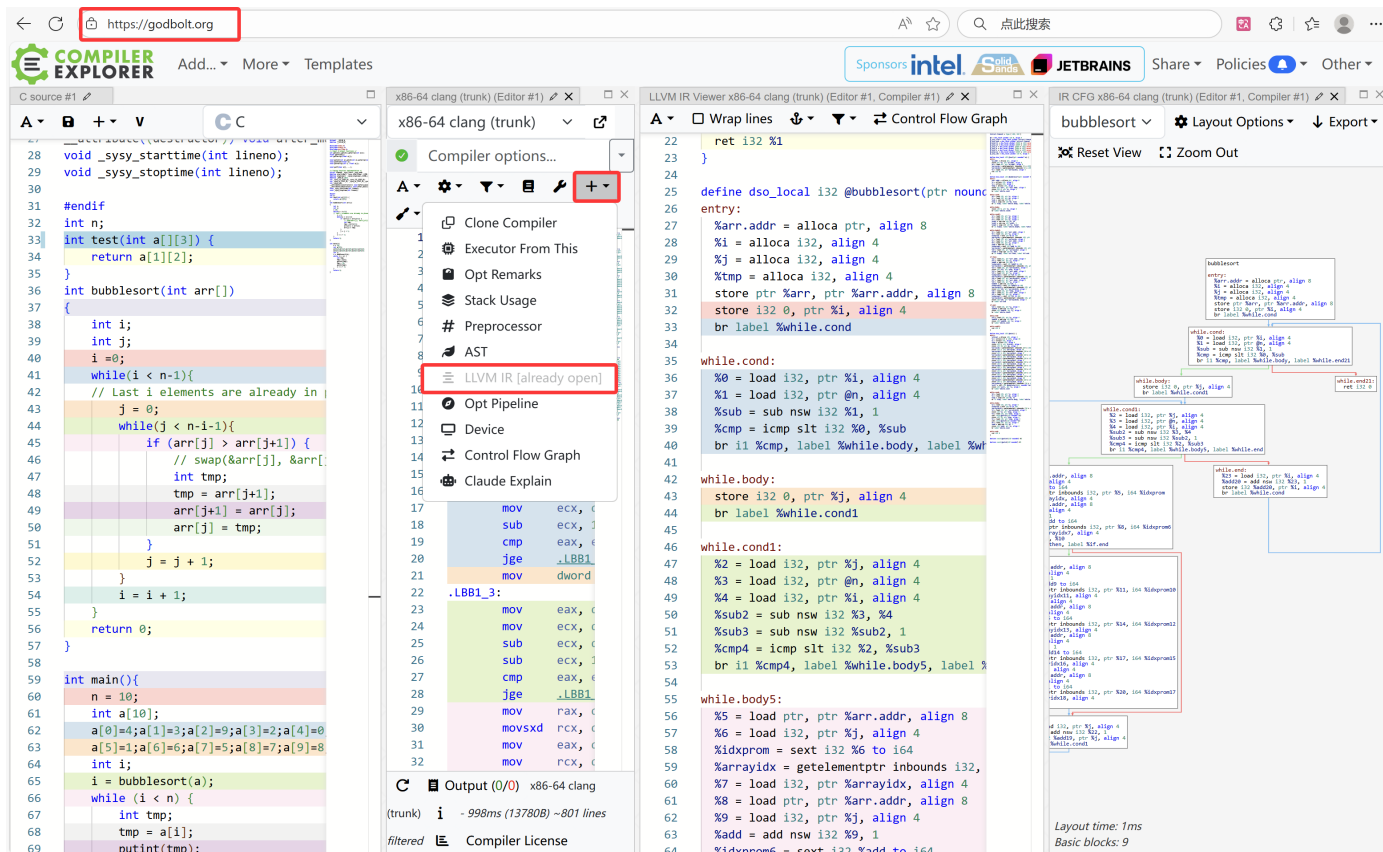
```
sudo apt update
# 在 PA3 中已经安装的部分
sudo apt install build-essential cmake git pkg-config python3 openjdk-11-jdk curl
# 在 大作业中新增的环境要求
sudo apt install clang
```

• 实用工具

对于一个 SysY 语言（C 语言），我要如何知道它的 LLVM IR 应该长什么样呢？一个方法是使用 `clang` 来获取 SysY 语言对应的 LLVM IR。

```
clang -S -emit-llvm -o test.ll test.c
cat test.ll
```

在这里，我们隆重介绍一个更好用的网站：[COMPILER EXPLORER](https://compiler-explorer.com/)。你可以选择不同的编译器，查看各种语言的编译结果。



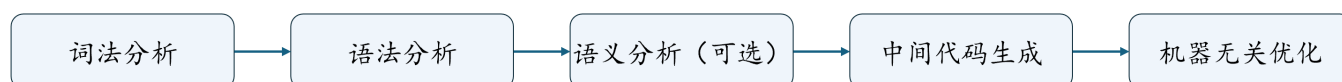
• LLVM 文档

- [LLVM Language Reference Manual](#)
- [LLVM Programmer's Manual](#)

• 编译器整体架构

在大作业中，我们仍然推荐使用现代的 ANTLR4 来完成词法分析和语法分析，这意味着你不需要手写词法分析器，以及具体实现 LL 或 LR 语法分析算法。

整体流程：



• 代码实现顺序 & 团队分工

小组可以根据团队的成员实际情况来决定如何分配工作。

在这里，助教给出一个可行的实现顺序：

1. 编译器软件架构设计
2. 词法分析 & 语法分析
3. 类型系统设计
4. IRBuilder、Instruction 类编写
5. main 函数和算术表达式 IR 生成
6. 符号表设计
7. 局部变量与全局变量的翻译
8. 控制流语句的翻译(if、while、break、continue)
9. 函数的翻译
10. 数组的翻译
11. 控制流图与机器无关优化 (Optional)

其中有很多步骤是可以并行进行的，例如不同 LLVM IR 指令的实现、不同功能 IR 的翻译等等，希望大家能合理分工大作业。

• 类型系统

为了类型检查以及描述 LLVM IR 中的类型信息，我们需要用特定的数据结构描述 SysY 语言中的类型。

如果语言中只存在 `int`、`void` 这样的基本类型，我们可以用一个简单的枚举来表示类型。

```
enum Type {  
    IntType,  
    VoidType  
};
```

但 SysY 语言中的类型要远比上面的基础类型复杂，例如数组类型中还需要考虑元素类型以及元素数量。因此，我们可以用一个类来表示具体的类型，这些类型都继承自一个抽象的 `Type` 基类。在将 SysY 语言翻译到 LLVM IR 的过程中，你至少需要考虑以下四种类型：

- 基本类型：例如 `int`、`void`
- 数组类型 `ArrayType`：例如 `int[10]`、`int[10][10]`，需要存储数组元素的类型以及元素数量
- 函数类型 `FunctionType`：例如 `int f(int a, int b)`，需要存储其返回类型以及每个参数的类型；在大作业中，返回值类型只能是基本类型

- 指针类型 `PointerType`：例如 `int *`、`int **`，需要存储指针指向的类型

下面是一个可供参考的设计，同学们可以自行设计类型系统的实现方式，但希望大家能采用良好的、具备可扩展性的软件设计，起码不要都用字符串来表示。

Type.h

```
/**
 * Array can be represented as [elementCount x elementType], e.g. [3 x
 i32].
 * elementType: the type of the array element
 * elementCount: represent the length of the array
 * e.g. [3 x [2 x i32]]: an array of length 3, the array element type
 is a 32-bit signed array of length 2
 * equivalent to int[3][2] in C / C++.
 */
class ArrayType : public Type {
public:
    ArrayType(Type *elementType, uint64_t elementCount)
        : Type(ArrayTypeID), elementType(elementType),
          elementCount(elementCount) {}

    std::string toString() const override;

    Type *getElementType() const { return elementType; }
    uint64_t getElementCount() const { return elementCount; }

private:
    Type *elementType;
    uint64_t elementCount;
};
```

在此之外，我们额外花一点篇幅讲一讲类型系统的注意点。

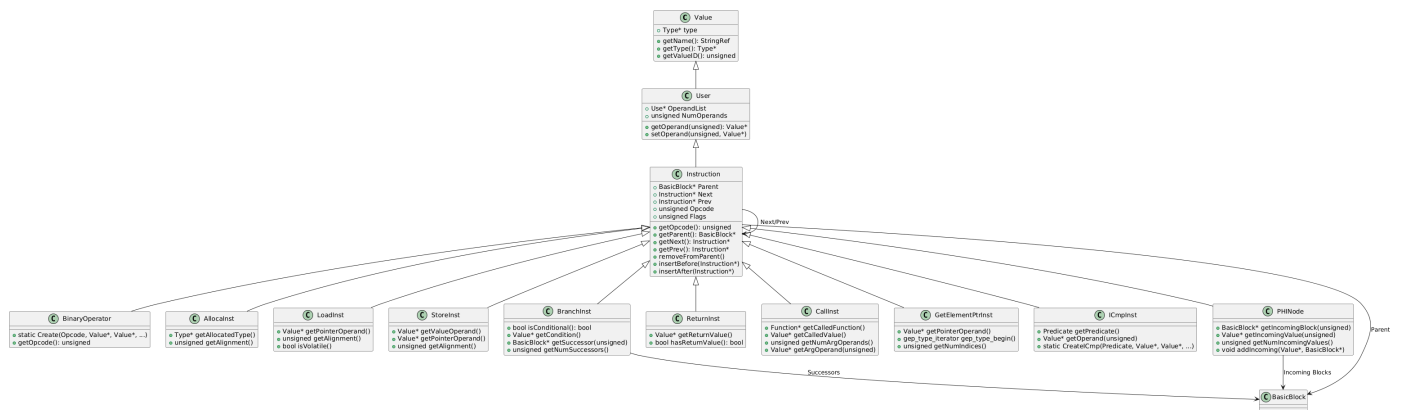
- 像基本类型 `int`、`void`，我们可以设计成 **单例模式** (Singleton Pattern)，避免创建重复的类型。`int` 类型可以存储一个参数表示的位数，例如 `int1`、`int32`、`int64`。
- 对于其他复杂的类型，其构造比较麻烦，我们可以用 **工厂** (Factory) 的思想来创建对象，在 C / C++ 中，你还需要考虑如何不产生内存泄漏。对于一个 C++ 程序员，只管 `new` 不管 `delete` 是一件很糟糕的事情。你可以参考 LLVM 的设计，也可以根据实际大作业的需求设计更简化的版本，例如使用智能指针管理内存。

• IRBuilder

IRBuilder 是 LLVM 中用于生成 IR 的类，它提供了一系列的函数，用于生成各种 IR 节点。虽然你可以直接在 **YourVisitor** 类遍历的过程中直接生成 LLVM IR 的字符串，就像在完成 PA3 时一样，但为了代码的可读性、可扩展性，我们建议你实现 **IRBuilder** 来生成内存形式的 IR。实际上，也只有这样，才能在后续构造控制流图、数据流图，进行中间代码优化以及编译器后端的指令选择、寄存器分配。添加一个 **IRBuilder** “中间层”的设计思想在复杂软件设计中非常重要，**IRBuilder** 也可以被视为 **建造者模式**（**Builder Pattern**）的一种实现。

（助教提供的文档只是一种可行的设计，你可以完全不遵守）

我们可以为 LLVM IR 中的不同指令设计一个类，例如 **ReturnInst**、**AllocaInst**、**StoreInst**、**LoadInst**、**BranchInst** 等等，这些类继承自一个基类 **Instruction**。下图大致描述了 LLVM 的设计，你可以适当参考。



LLVM Instruction Design (AI 生成)

这里助教偷懒一下，让大语言模型来介绍一下 LLVM 是怎么设计的。不过你不必完全按照 LLVM 的设计来，毕竟我们要实现的东西比真实编译器还是简单得多。

为我介绍 LLVM 中 **IRBuilder** 的设计

你的设计不必与 LLVM 相同，你可以自由设计 **Instrcution** 的派生类，例如考虑设计不同的 **AddInstruction** 和 **SubInstruction** 还是将其统一为 **CalcInstruction**，甚至只有一个 **Instruction** 类，通过 **InstType** 来区分不同的指令。但是我们不推荐你直接使用字符串来表示指令，这不符合面向对象的封装逻辑，也对后续编译优化和项目维护带来困难。

除了上面大语言模型对 LLVM **IRBuilder** 的介绍外，你还可以参考 LLVM Java API 的设计。希望这些内容对大家设计编译器有所帮助。

LLVM IRBuilder Java API

```
// 创建module
LLVMModuleRef module = LLVMModuleCreateWithName("module");
```

```

// 创建IRBuilder
LLVMBuilderRef builder = LLVMCreateBuilder();

// 获取int32基本类型
LLVMTypeRef i32Type = LLVMInt32Type();

//创建一个常量,这里是常数0
LLVMValueRef zero = LLVMConstInt(i32Type, 0, /* signExtend */ 0);

//创建名为globalVar的全局变量
LLVMValueRef globalVar = LLVMAddGlobal(module, i32Type,
/*globalVarName:String*/"globalVar");

//为全局变量设置初始化器
LLVMSetInitializer(globalVar, /* constantVal:LLVMValueRef*/zero);

//生成返回值类型
LLVMTypeRef returnType = i32Type;

//生成函数参数类型
PointerPointer<Pointer> argumentTypes = new PointerPointer<>(2)
    .put(0, i32Type)
    .put(1, i32Type);

//生成函数类型
LLVMTypeRef ft = LLVMFunctionType(returnType, argumentTypes, /*
argumentCount */ 2, /* isVariadic */ 0);
//若仅需一个参数也可以使用如下方式直接生成函数类型
ft = LLVMFunctionType(returnType, i32Type, /* argumentCount */ 1, /*
isVariadic */ 0);

//生成函数,即向之前创建的module中添加函数
LLVMValueRef function = LLVMAddFunction(module,
/*functionName:String*/"function", ft);

//通过如下语句在函数中加入基本块,一个函数可以加入多个基本块
LLVMBasicBlockRef block1 = LLVMAppendBasicBlock(function,
/*blockName:String*/"block1");

LLVMBasicBlockRef block2 = LLVMAppendBasicBlock(function,
/*blockName:String*/"block2");

//选择要在哪个基本块后追加指令

```

```

LLVMPositionBuilderAtEnd(builder, block1); //后续生成的指令将追加在block1的
后面

//获取函数的参数
LLVMValueRef n = LLVMGetParam(function, /* parameterIndex */0);

//创建add指令并将结果保存在一个临时变量中
LLVMValueRef result = LLVMBuildAdd(builder, n, zero, /* varName:String
*/"result");

//跳转指令决定跳转到哪个块
LLVMBuildBr(builder, block2);

//生成比较指令
LLVMValueRef condition = LLVMBuildICmp(builder, /*这是个int型常量, 表示比
较的方式*/LLVMIntEQ, n, zero, "condition = n == 0");
/* 上面参数中的常量包含如下取值
    LLVMIntEQ,
    LLVMIntNE,
    LLVMIntUGT,
    LLVMIntUGE,
    LLVMIntULT,
    LLVMIntULE,
    LLVMIntSGT,
    LLVMIntSGE,
    LLVMIntSLT,
    LLVMIntSLE,
*/
//条件跳转指令, 选择跳转到哪个块
LLVMBuildCondBr(builder,
/*condition:LLVMValueRef*/ condition,
/*ifTrue:LLVMBasicBlockRef*/ ifTrue,
/*ifFalse:LLVMBasicBlockRef*/ ifFalse);

LLVMPositionBuilderAtEnd(builder, block2); //后续生成的指令将追加在block2的
后面

//函数返回指令
LLVMBuildRet(builder, /*result:LLVMValueRef*/result);

LLVMPositionBuilderAtEnd(builder, block1);

//int型变量
//申请一块能存放int型的内存

```

```

LLVMValueRef pointer = LLVMBuildAlloca(builder, i32Type,
/*pointerName:String*/"pointer");

//将数值存入该内存
LLVMBuildStore(builder, zero, pointer);

//从内存中将值取出
LLVMValueRef value = LLVMBuildLoad(builder, pointer,
/*varName:String*/"value");

//数组变量
//创建可存放200个int的vector类型
LLVMTypeRef vectorType = LLVMVectorType(i32Type, 200);

//申请一个可存放该vector类型的内存
LLVMValueRef vectorPointer = LLVMBuildAlloca(builder, vectorType,
"vectorPointer");

```

• 符号表

在 PA4 中，你已经设计了一个符号表，在大作业中的符号表与 PA4 中的符号表类似，都需要考虑作用域的问题。不过，你需要考虑得更细致一些，进入函数、if语句等节点中，作用域都会改变。

符号表里存什么呢？你需要存 LLVM 中的 Value，还可能要存一些类型信息，这个就交给你自己设计啦。

• 指令翻译

推荐使用 Visitor 模式遍历 ANTLR4 生成的语法树，并在 Visitor 中完成指令翻译。

控制流语句翻译：你需要考虑短路求值